WEEK 9 – SALES AND CUSTOMER ANALYTICS SYSTEM

1. Introduction

The Sales and Customer Analytics System is used to analyse the sales data of an e-commerce company. It helps understand customer purchasing behaviour, product performance, and revenue trends. By analysing customer orders and product sales, the management team can identify top customers, best-selling products, and profitable categories. This system supports better business planning and decision-making.

2. Objective

The main objectives of this system are:

- To apply SQL aggregate functions for analysing sales data.
- To calculate total revenue, number of orders, and average order value.
- To identify customers based on their purchase amount and order frequency.
- To find best-selling and least-selling products.
- To perform category-wise sales analysis.
- To generate business analytics reports for management.

3. SQL Aggregate Functions

SQL aggregate functions are used to perform calculations on multiple records and return summarised results. These functions help analyse sales and customer information efficiently.

- **COUNT():** Counts the total number of records or orders.
- **SUM():** Calculates the total sales amount or revenue.
- **AVG():** Calculates the average order value or customer spending.
- **MIN():** Identifies the lowest order amount or product price.
- **MAX():** Identifies the highest order amount or product price.

4. Total Sales Analysis

Total sales analysis provides information about the company's sales performance. SQL queries are used to calculate the total number of orders placed, total revenue generated, average order value, and highest and lowest order amounts. Sales can also be analysed for a specific period to understand changes in revenue.

1.Total number of orders placed.

```
SQL> select count(*) AS Numbers_of_placed from orders;

NUMBERS_OF_PLACED
-----------------
                9
```

2.Total revenue generated from sales.

```
SQL> SELECT SUM(total_amount) AS total_revenue FROM orders;

TOTAL_REVENUE
-------------
        34300
```

3.Average order value.

```
SQL> SELECT AVG(total_amount) AS average_order_value FROM orders;

AVERAGE_ORDER_VALUE
-------------------
         3811.11111
```

4.Highest and lowest order amount.

```
SQL> SELECT MAX(total_amount) AS highest_order, MIN(total_amount) AS lowest_order FROM orders;

HIGHEST_ORDER LOWEST_ORDER
------------- ------------
         7500         1600
```

5.Total sales for a specific period.

```
SQL> SELECT SUM(total_amount) AS period_sales FROM orders WHERE order_date >= DATE '2026-09-01' AND order_date < DATE '2026-10-01';

PERIOD_SALES
------------
       34300
```

5. Customer Purchase Analysis

Customer purchase analysis helps understand the purchasing behaviour of individual customers. It calculates the total number of orders placed by each customer, total amount spent, and average spending. It also identifies customers with maximum purchase amounts and customers who have made fewer purchases.

1. Total number of orders placed by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GR
OUP BY c.customer_id, c.customer_name;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS
------------
       103
MANOJ
           2

       102
kavya
           2

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS
------------

       104
NISHA
           1

       101
AJAY
```

2. Total amount spent by each customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(SUM(o.total_amount), 0) AS total_spent FROM customer c LEFT JOIN orders o ON c.customer_id = o.custo
mer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
        103
MANOJ
       4800

        102
kavya
       8600

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------

        104
NISHA
       2800

        101
AJAY
```

The NVL ( ) function can be used to display zero for customers who have no purchases instead of showing a NULL value.

3. Average spending per customer.

```
SQL> SELECT c.customer_id, c.customer_name, NVL(AVG(o.total_amount), 0) AS average_spending FROM customer c LEFT JOIN orders o ON c.customer_id = o.
customer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
AVERAGE_SPENDING
----------------
        103
MANOJ
            2400

        102
kavya
            4300

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
AVERAGE_SPENDING
----------------

        104
NISHA
            2800

        101
AJAY
```

4. Customers with maximum purchase amount.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) = (SELECT MAX(customer_total) FROM (SELECT SUM(total_amount) AS customer_total FROM ord
ers GROUP BY customer_id));

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_SPENT
-----------
        105
RAVI
       12700
```

5. Customers with fewer purchases.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c LEFT JOIN orders o ON c.customer_id = o.customer_id GR
OUP BY c.customer_id, c.customer_name HAVING COUNT(o.order_id) <= 1;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS
------------
        104
NISHA
           1
```

6. Top Customer Analysis

Top customer analysis identifies customers who contribute significantly to the company's revenue. Customers can be ranked based on their total spending and the number of orders placed. Frequent customers and high-value customers can also be identified through SQL queries.

1. Top 5 customers based on total spending.

```
SQL> SELECT * FROM (SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.cus
tomer_id GROUP BY c.customer_id, c.customer_name ORDER BY total_spent DESC) WHERE ROWNUM <= 5;

CUSTOMER_ID CUSTOMER_NAME                                                                                          TOTAL_SPENT
----------- -------------------------------------------------------------------------------------------------- -----------
        105 RAVI                                                                                                     12700
        102 kavya                                                                                                     8600
        101 AJAY                                                                                                      5400
        103 MANOJ                                                                                                     4800
        104 NISHA                                                                                                     2800
```

2. Customers with maximum number of orders.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP B
Y c.customer_id, c.customer_name HAVING COUNT(o.order_id) = (SELECT MAX(order_count) FROM (SELECT COUNT(*) AS order_count FROM orders GROUP BY custo
mer_id));

CUSTOMER_ID CUSTOMER_NAME                                                                                          TOTAL_ORDERS
----------- -------------------------------------------------------------------------------------------------- ------------
        103 MANOJ                                                                                                         2
        102 kavya                                                                                                         2
        101 AJAY                                                                                                          2
        105 RAVI                                                                                                          2
```

3. Frequent customers.

```
SQL> SELECT c.customer_id, c.customer_name, COUNT(o.order_id) AS total_orders FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP B
Y c.customer_id, c.customer_name HAVING COUNT(o.order_id) >= 2;

CUSTOMER_ID CUSTOMER_NAME                                                                                          TOTAL_ORDERS
----------- -------------------------------------------------------------------------------------------------- ------------
        103 MANOJ                                                                                                         2
        102 kavya                                                                                                         2
        101 AJAY                                                                                                          2
        105 RAVI                                                                                                          2
```

4. High-value customers.

```
SQL> SELECT c.customer_id, c.customer_name, SUM(o.total_amount) AS total_spent FROM customer c JOIN orders o ON c.customer_id = o.customer_id GROUP
BY c.customer_id, c.customer_name HAVING SUM(o.total_amount) >= 50000 ORDER BY total_spent DESC;

no rows selected
```

This information helps the company understand customer loyalty and plan suitable offers to encourage repeat purchases.

7.Best -Selling Product Performance Analysis

Product performance analysis is used to evaluate how well each product performs in the market. SQL queries help identify products with the highest sales quantity, products generating the highest revenue, and products with fewer sales.

The results help management understand customer preferences and identify products that may require promotional offers or marketing activities.

1. Products with maximum sales quantity.

```
SQL> SELECT p.product_id, p.product_name, SUM(od.quantity) AS quantity_sold FROM product p JOIN order_details od ON p.product_id = od.product_id GRO
UP BY p.product_id, p.product_name HAVING SUM(od.quantity) = (SELECT MAX(total_quantity) FROM (SELECT SUM(quantity) AS total_quantity FROM order_det
ails GROUP BY product_id));

PRODUCT PRODUCT_NAME         QUANTITY_SOLD
------- -------------------- -------------
P305    Room 301                         4
```

2. Products generating highest revenue.

```
SQL> SELECT p.product_id, p.product_name, SUM(od.quantity * od.price) AS revenue FROM product p JOIN order_details od ON p.product_id = od.product_i
d GROUP BY p.product_id, p.product_name ORDER BY revenue DESC;

PRODUCT PRODUCT_NAME            REVENUE
------- -------------------- ----------
P310    Room 601                   7500
P305    Room 301                   5200
P306    Room 302                   3600
P303    Room 201                   3200
P309    Room 501                   2800
P302    Room 102                   2550
P301    Room 101                   1800
P308    Room 402                   1600

8 rows selected.
```

3. Least-selling products.

```
SQL> SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.p
roduct_id GROUP BY p.product_id, p.product_name ORDER BY quantity_sold ASC;

PRODUCT PRODUCT_NAME         QUANTITY_SOLD
------- -------------------- -------------
P304    Room 202                         0
P311    ROOM 61                          0
P303    Room 201                         1
P308    Room 402                         2
P301    Room 101                         2
P309    Room 501                         2
P306    Room 302                         2
P302    Room 102                         3
P310    Room 601                         3
P305    Room 301                         4

10 rows selected.
```

4. Products requiring promotion.

```
SQL> SELECT p.product_id, p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold FROM product p LEFT JOIN order_details od ON p.product_id = od.p
roduct_id GROUP BY p.product_id, p.product_name HAVING NVL(SUM(od.quantity), 0) <= 1;

PRODUCT PRODUCT_NAME         QUANTITY_SOLD
------- -------------------- -------------
P311    ROOM 61                          0
P303    Room 201                         1
P304    Room 202                         0
```

8. Category-wise Sales Analysis

Category-wise sales analysis evaluates the performance of different product categories. It calculates the total sales generated by each category, the number of products sold, and the average sales per category. It also identifies the category that generates the highest revenue.

This analysis helps management compare categories and make better decisions regarding product selection and sales strategies.

1. Total sales generated by each category.

```
SQL> SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_sales FROM category c LEFT JOIN product p ON c.category_id
= p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name;

CATEGOR CATEGORY_NAME                                                                                    TOTAL_SALES
------- ---------------------------------------------------------------------------------------------- -----------
309     Twin                                                                                              2800
306     Family                                                                                            3600
310     Presidential                                                                                      7500
304     Suite                                                                                                0
303     Deluxe                                                                                            3200
307     Premium                                                                                              0
301     Single                                                                                            1800
302     Double                                                                                            2550
305     Executive                                                                                         5200
308     Economy                                                                                           1600

10 rows selected.
```

2. Number of products sold in each category.

```
SQL>  SELECT c.category_name, NVL(SUM(od.quantity), 0) AS products_sold FROM category c LEFT JOIN product p ON c.category_id = p.category_id LEFT JO
IN order_details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME                                                                                    PRODUCTS_SOLD
------------------------------------------------------------------------------------------------ -------------
Family                                                                                                      2
Economy                                                                                                     2
Suite                                                                                                       0
Single                                                                                                      2
Deluxe                                                                                                      1
Executive                                                                                                   4
Twin                                                                                                        2
Presidential                                                                                                3
Premium                                                                                                     0
Double                                                                                                      3

10 rows selected.
```

3. Highest revenue-generating category.

```
SQL> SELECT * FROM (SELECT c.category_id, c.category_name, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c LEFT JOIN product p
ON c.category_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_name ORDER BY total_r
evenue DESC) WHERE ROWNUM = 1;

CATEGOR CATEGORY_NAME                                                                                  TOTAL_REVENUE
------- ---------------------------------------------------------------------------------------------- -------------
310     Presidential                                                                                          7500
```

4. Average sales per category.

```
SQL> SELECT c.category_name, NVL(AVG(od.quantity * od.price), 0) AS average_sales FROM category c LEFT JOIN product p ON c.category_id = p.category_
id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_name;

CATEGORY_NAME                                                                                    AVERAGE_SALES
------------------------------------------------------------------------------------------------ -------------
Family                                                                                                   3600
Economy                                                                                                  1600
Suite                                                                                                       0
Single                                                                                                   1800
Deluxe                                                                                                   3200
Executive                                                                                                5200
Twin                                                                                                     2800
Presidential                                                                                             7500
Premium                                                                                                     0
Double                                                                                                   2550

10 rows selected.
```

9. Business Analytics Reports

Business analytics reports present important sales and customer information in a structured format. These reports help management understand the company's overall performance.

Report 1: Sales Performance Report

- Total sales

- Number of orders
- Average order value
- Highest order value

```
SQL> SELECT COUNT(*) AS number_of_orders, SUM(total_amount) AS total_sales, AVG(total_amount) AS average_order_value, MAX(total_amount) AS highest_o
rder_value FROM orders;

NUMBER_OF_ORDERS TOTAL_SALES AVERAGE_ORDER_VALUE HIGHEST_ORDER_VALUE
---------------- ----------- ------------------- -------------------
               9       34300          3811.11111                7500
```

Report 2: Customer Analytics Report

- Customer name
- Number of orders
- Total spending
- Purchase frequency

```
SQL> SELECT c.customer_name, COUNT(o.order_id) AS number_of_orders, NVL(SUM(o.total_amount), 0) AS total_spending FROM customer c LEFT JOIN orders o
 ON c.customer_id = o.customer_id GROUP BY c.customer_id, c.customer_name;

CUSTOMER_NAME                                                                                      NUMBER_OF_ORDERS TOTAL_SPENDING
-------------------------------------------------------------------------------------------------- ---------------- --------------
MANOJ                                                                                                             2           4800
kavya                                                                                                             2           8600
NISHA                                                                                                             1           2800
AJAY                                                                                                              2           5400
RAVI                                                                                                              2          12700
```

Report 3: Product Performance Report

- Product name
- Quantity sold
- Revenue generated

```
SQL> SELECT p.product_name, NVL(SUM(od.quantity), 0) AS quantity_sold, NVL(SUM(od.quantity * od.price), 0) AS revenue_generated FROM product p LEFT
JOIN order_details od ON p.product_id = od.product_id GROUP BY p.product_id, p.product_name ORDER BY revenue_generated DESC;

PRODUCT_NAME         QUANTITY_SOLD REVENUE_GENERATED
-------------------- ------------- -----------------
Room 601                         3              7500
Room 301                         4              5200
Room 302                         2              3600
Room 201                         1              3200
Room 501                         2              2800
Room 102                         3              2550
Room 101                         2              1800
Room 402                         2              1600
ROOM 61                          0                 0
Room 202                         0                 0

10 rows selected.
```

Report 4: Category Analysis Report

- Category name
- Total products sold
- Total revenue

```
SQL> SELECT c.category_name, NVL(SUM(od.quantity), 0) AS total_products_sold, NVL(SUM(od.quantity * od.price), 0) AS total_revenue FROM category c L
EFT JOIN product p ON c.category_id = p.category_id LEFT JOIN order_details od ON p.product_id = od.product_id GROUP BY c.category_id, c.category_na
me ORDER BY total_revenue DESC;

CATEGORY_NAME                                                                                                    TOTAL_PRODUCTS_SOLD TOTAL_REVENUE
---------------------------------------------------------------------------------------------------------------- ------------------- -------------
Presidential                                                                                                                   3          7500
Executive                                                                                                                      4          5200
Family                                                                                                                         2          3600
Deluxe                                                                                                                         1          3200
Twin                                                                                                                           2          2800
Double                                                                                                                         3          2550
Single                                                                                                                         2          1800
Economy                                                                                                                        2          1600
Premium                                                                                                                        0             0
Suite                                                                                                                          0             0

10 rows selected.
```

10. Result

SQL aggregate functions and analytical queries were used to examine sales data, customer purchases, product performance, and category-wise revenue. Business analytics reports were prepared to summarise important information.

11. Conclusion

Thus, the Sales and Customer Analytics System was studied and implemented using SQL queries. It helps analyse customer purchasing behaviour, identify best-selling products, calculate revenue, and evaluate category performance. The generated reports support effective business planning and better management decisions.